Lab 12 — Memory Vulnerabilities & Exploit Mitigations

> **SY0-701 Objective 2.3** — Explain various types of vulnerabilities
>
> **Domain 2** — Threats, Vulnerabilities, and Mitigations (22%) · **Time:** ~25 min · **Course:** TGS-2023039181 CompTIA Security+ (SY0-701)

Goal: make a tiny C program crash from a stack-based buffer overflow, then rebuild it so the compiler's memory protections stop the same input.

You'll build: a before/after pair of binaries that demonstrates the buffer-overflow vulnerability class and the mitigations (stack canary, FORTIFY, PIE, ASLR) that defeat it.

Prerequisites: Killercoda Ubuntu Playground (root shell) — https://killercoda.com/playgrounds/scenario/ubuntu ; tools installed in step 1. All code is synthetic and runs only on your own playground.

Background

- A **buffer overflow** writes past the end of a fixed-size buffer, overwriting adjacent memory such as the saved return address, which can let an attacker redirect execution.
- **Memory injection** is the broader class: hostile data or code placed into a running process's memory to hijack it.
- A **race condition** (time-of-check to time-of-use, TOCTOU) is a related memory/timing flaw where state changes between validation and use.
- Modern toolchains ship mitigations: **stack canaries** (`-fstack-protector-strong`), **FORTIFY** bounds checks (`-D_FORTIFY_SOURCE=2`), **PIE/ASLR** (position-independent code plus address randomisation), and non-executable stacks.
- The exam wants you to recognise the vulnerability by name and know that these compiler/OS defences are the mitigation.

Steps

1. Install the tools

Install the GNU C compiler, debugger and ELF tools.

```
apt-get update -qq && apt-get install -y gcc gdb binutils python3
```

2. Write a tiny vulnerable program

`strcpy` copies attacker-controlled input into a 16-byte buffer with no bounds check — the classic overflow.

```
cat > vuln.c <<'EOF'
#include <stdio.h>
#include <string.h>
int main(int argc, char **argv) {
```

```
    char buf[16];
    if (argc < 2) { printf("usage: %s <input>\n", argv[0]); return 1; }
    strcpy(buf, argv[1]);      /* no bounds check: overflow */
    printf("stored: %s\n", buf);
    return 0;
}
EOF
```

3. Compile it with protections disabled

Turn off the canary, PIE and the non-executable stack so the overflow is observable.

```
gcc -fno-stack-protector -z execstack -no-pie -O0 -o vuln_off vuln.c
file vuln_off
```

4. Trigger the overflow crash

A short input is fine; a long input overruns the buffer and corrupts the stack, causing a segmentation fault.

```
./vuln_off hello; echo "short exit=$?"
./vuln_off "$(python3 -c 'print("A"*64)')"; echo "long exit=$?"
```

```
short exit=0
Segmentation fault (core dumped)
long exit=139        # 128+SIGSEGV(11) = stack corrupted
```

5. Check the system ASLR setting

Address Space Layout Randomisation makes exploitation harder by randomising memory addresses. `2` means full randomisation.

```
cat /proc/sys/kernel/randomize_va_space
```

```
2
```

6. Rebuild with the mitigations on

Enable the stack canary, FORTIFY bounds checks and PIE. These are the defaults on modern Ubuntu; here we set them explicitly.

```
gcc -fstack-protector-strong -D_FORTIFY_SOURCE=2 -O2 -pie -fPIE -o vuln_on vuln.c
```

7. Confirm the canary now catches the overflow

The same long input now aborts cleanly with a detection message instead of silently corrupting memory.

```
./vuln_on "$(python3 -c 'print("A"*64)')" 2>&1 | head -2; echo "exit=${PIPESTATUS[0]}"
```

```
*** stack smashing detected ***: terminated
Aborted (core dumped)
```

8. Verify the binary's protections

`PIE` (position-independent) plus a `__stack_chk` symbol confirm the hardening is compiled in.

```
file vuln_on
readelf -d vuln_on 2>/dev/null | grep -i 'bind\|now' || true
nm vuln_on 2>/dev/null | grep -i stack_chk || echo "canary symbol resolved at runtime"
```

Test it

Confirm the two builds behave differently on the same hostile input.

```
./vuln_off "$(python3 -c 'print("A"*64)')" 2>/dev/null; OFF=$?
./vuln_on  "$(python3 -c 'print("A"*64)')" 2>/dev/null; ON=$?
echo "unprotected=$OFF protected=$ON"
[ "$OFF" -ge 128 ] && [ "$ON" -ge 128 ] && echo "BOTH CRASH, but protected one printed
'stack smashing detected' = mitigation works"
```

Expected: both exit codes are ≥128 (crash), but only the unprotected build silently corrupts the stack; the protected build aborts via the canary.

Troubleshooting

Symptom	Fix
`gcc: command not found`	Re-run step 1; on minimal images also `apt-get install -y build-essential`.
Unprotected build does not crash	Increase the input length, e.g. `"A"*128`; stack layout varies by compiler version.
`-z execstack` warning	Harmless on modern gcc; the overflow still demonstrates.

Challenge

Add a **TOCTOU race condition** demo: write a script that checks a file with `[ -O file ]` then acts on it, and show how swapping the file (symlink) between the check and the use defeats the check. Explain why atomic operations fix it.

Reflection

Objective 2.3 lists buffer overflow, memory injection and race conditions as application vulnerabilities. Why is a compiler/OS mitigation (canary, ASLR, PIE) a *defence-in-depth* layer rather than a fix for the underlying `strcpy` bug?

Exam tips

- Buffer overflow overwrites adjacent memory (often the return address); memory injection writes code into a running process.
- Race condition keyword is **TOCTOU** — time-of-check to time-of-use.

- Mitigations: stack canaries, ASLR, DEP/NX (non-executable stack), and safe functions (`strncpy`, bounds checks).
- The real fix is secure coding and input validation; the compiler flags only make exploitation harder.

References

- OWASP Buffer Overflow — https://owasp.org/www-community/vulnerabilities/Buffer_Overflow
- CWE-121 Stack-based Buffer Overflow — https://cwe.mitre.org/data/definitions/121.html
- CWE-367 TOCTOU Race Condition — https://cwe.mitre.org/data/definitions/367.html
- Ubuntu Security Features (compiler hardening) — https://wiki.ubuntu.com/Security/Features

---